Gagandeep Bhatia

Term Project: Instant Messaging App

Architecture Document

Table of Contents

1 Introduction 2

1.1 Purpose 2

1.2 Target Audience 2

1.3 Terms and Definitions 2

2 Architectural Goals and Constraints 4

2.1 Security 4

2.2 Persistency 4

2.3 Reliability/Availability 4

3 System Overview 5

4 System Architecture 5

4.1 Front-end Architecture 7

4.2 Back-end Microservices Architecture 9

5 Database Design 11

6 Issue and Concerns 12

1 Introduction

This is the architecture document for the instant messaging app. The document covers the architecture goals and constraints, an overview of the system, the system architecture, database design and issues that have arisen thus far.

1.1 Purpose

Provide a high-level description of the purpose of this document. How will this document help you or others when developing similar product?

This document is to be used as a high-level overview of the architecture of this instant messaging app. It covers the goals of the system, a high-level overview of the system at each layer from the user interface layer all the way to the database management layer. It also describes the frontend, backend and database architecture in detail.

1.2 Target Audience

The target audience for this document is myself, the TA Huan Wu, and Professor Fei Xie.

1.3 Terms and Definitions

Define any terms or acronyms you will be using in the remainder of this document.

BSON: Binary JSON

JSON: JavaScript Object Notation

JavaScript Object Notation: Human readable format for storing and exchanging data between a server and a web application.

JWT: JSON Web Tokens which are used to do user authentication/authorization.
Socket.IO: A library for real-time WebSocket communication.

WebSocket: Allows a client and server to both send and receive messages simultaneously.

AyncIO: Waits for a response to come back, does not need the data/response instantly. This allows other processes to run while the current process waits for the data/response it needs.

2 Architectural Goals and Constraints

2.1 Security

The systems uses JWT (JSON Web Tokens) to authenticate and authorize users. When a user signs up the accepted input is an email and password, the password is then then hashed with bcrypt, and then a JWT token is generated and stored as a cookie. The other route is for when the user logs in, it verifies the user credentials and then creates and stores a JWT cookie on a successful login. All subsequent routes use a verifyToken middleware to ensure that only requests containing a valid JWT cookie can access protected endpoints.

2.2 Persistency

The system achieves data persistence by using a MongoDB database to store user data such as credentials, messaging history, chat rooms, and contacts.

2.3 Reliability/Availability

The main constraint/reliability of the system features is relying on Ngrok to access the remote backend. If developers forget to include “ngrok-skip-browser-warning” : “true when configuring Axios headers in the frontend, this could lead to the application not acting as expected due to CORS errors and Ngrok security splash pages blocking connections.

3 System Overview

The instant messaging apps user interface is conducted via a web browser and built in React. The frontend manages the user interface via different options for users such as signup, login, modify profile, chat with a contact directly, chat with a group of users, and delete specific chats. One service the application provides to users is logging up and signing into the platform via an email and password. Another service the application provides is the ability to create chat rooms with either one or many participants. The application allows users send and receive messages in real-time via asyncIO. It also allows users to add and remove participants from chat rooms after the room has been created. Additionally, the application also allows the user to block other users, assigning owners to multiple chats, and editing/deleting messages once they have been sent. The application also allows users to customize their profile.

The system also uses a number of shared infrastructure services. The system uses authentication through JWT tokens when users signup/login and a verifyToken on all subsequent requests once a user has logged in. Another shared infrastructure service is AWS to host the frontend and backend and also each microservice to it's own virtual server with instances being launched or deprecated as needed based on user demand. The system also uses WebSockets for instant messaging between users via asyncIO. The system also logs user actions and stores user data on MongoDB, and the database is hosted on MongoDB Atlas.

In summary, the architectural model of the system begins with the first/top-most layer with a browser based interface with buttons for the instant messaging app. At the second layer authentication occurs via JWT tokens and verifyToken's. At the third layer the interaction between the user and the application is managed through the frontend and requested calls are sent to the backend. The fourth layer consists of functionality to this chat app such as creating chat rooms and real-time communication via WebSockets and asyncIO. The fifth layer is all shared services across the app, and this consists of

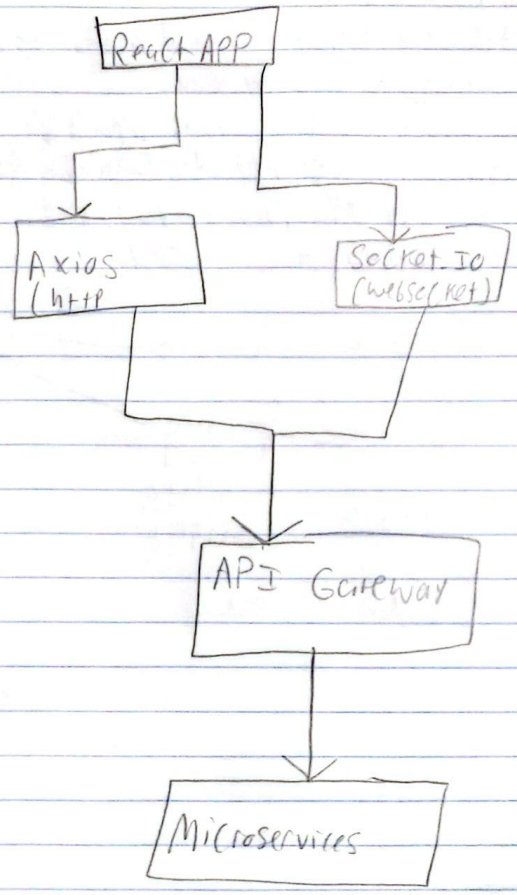
authentication, hosting through AWS or another cloud provider, and WebSockets. The sixth layer is database management which is conducted via MongoDB and hosted on MongoDB atlas.

4 System Architecture

4.1 Front-end Architecture

The frontend is created using React. It integrates APIs via axios and real-time chat is done using SocketIO through asyncIO. Thereafter requests are sent to SocketIO which then routes requests to the backend. There is an API gateway which sends the request to the microservice which has implemented that logic. The frontend APIs make the initial request for data or functionality to the backend. The core/main frontend APIs are for authentication, a user's contacts, and retrieving their messages. Axios is used to make REST requests. The frontend also displays automatic timeouts and integrates authentication via JWT tokens and verifyTokens. It also uses multi-factor authentication to ensure that users are authenticated and to limit the number of malicious users. The primary cloud/platform software is AWS to host the application and instances of the front-end can be spun up as needed per user demand.

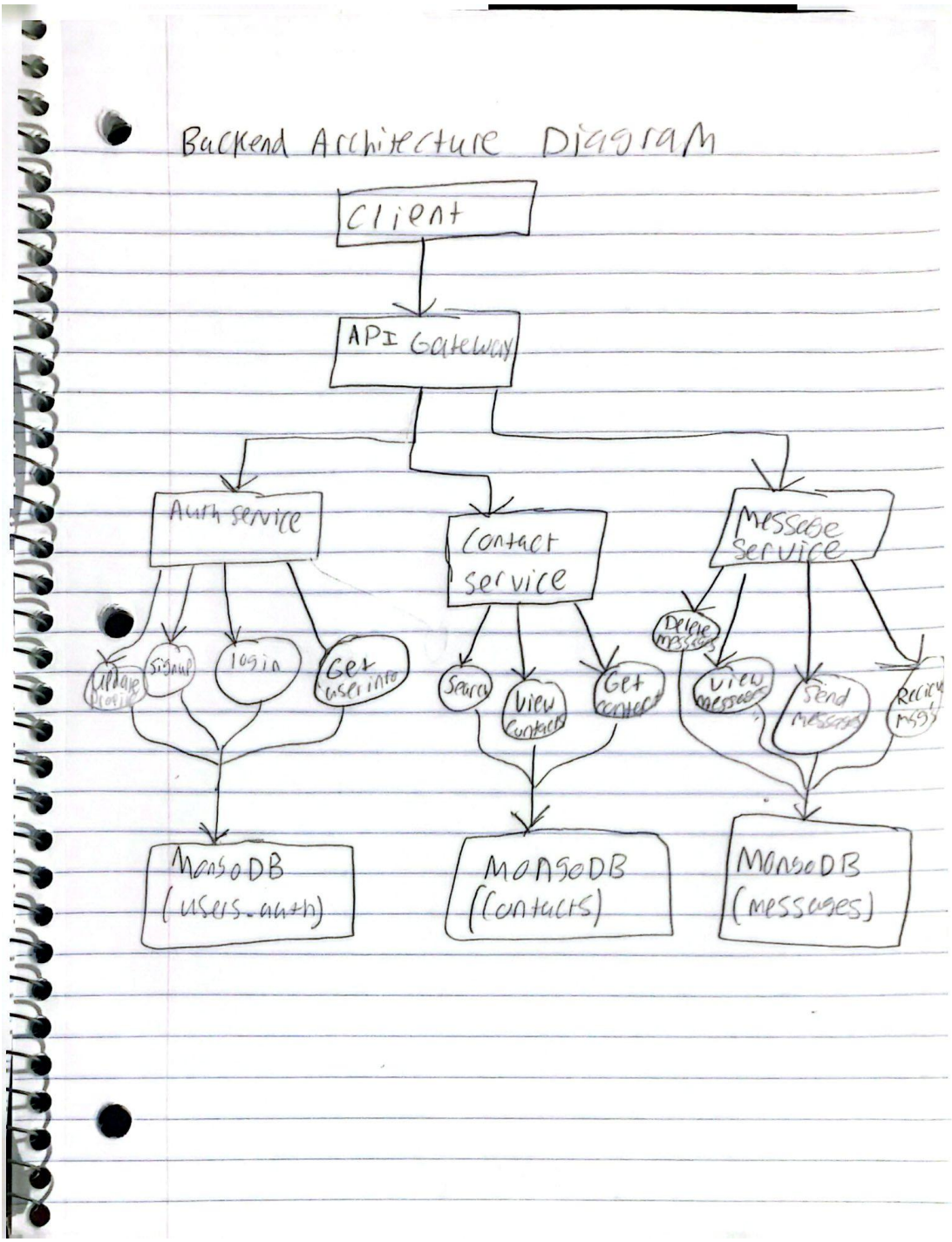
Architecture Document Diagrams



4.2 Back-end Microservices Architecture

The backend framework is Node.js and Express for creating RESTful APIs. The API routes are divided into 3 main components: authentication routes, contact routes and message routes. By following microservices architecture principles we can create smaller microservices until each microservice only handles one specific task. In authentication a user must signup, login, logout, get user info and update profile. Once these core functionalities are uncovered we can make each of these their own microservice and thus they are self-contained, they are not dependent on any other microservice and each of these microservices only know about the relevant data to the task, not about data for other microservices. Similarly we can apply the same principles to when a user wants to contact and message someone. When a user wants to contact someone they can: search for someone, view their contact list, and get a specific contact. When a user wants to message someone they can: view messages, send messages, receive messages, and delete messages. Each of these functionalities/features within each specific area is a microservice.

There is an API gateway which is routing requests to the correct microservice and then the microservice performs the necessary actions and queries the database for the result and returns it back to the calling/requesting service. Microservices communicate with one another using a message broker. The cloud-based software which I plan to use is AWS and the database is orchestrated through MongoDB and hosted via MongoDB Atlas.


Backend Architecture Diagram
Client
API Gateway
Auth service
Contact service
Message service
Update profile
Signup
login
Get user info
Search
View contacts
Get contact
Delete messages
View messages
Send messages
Recieve msgs
MongoDB (users_auth)
MongoDB (contacts)
MongoDB (messages)

5 Database Design

This section should describe the design of the database or data hosting environment. You should include which database you have chosen and what is the advantage of using it. Describe your database design and tell us how the design achieves interoperability between the user interface and the background data.
The database design should include all the data utilized by your product.

The database I have chosen is MongoDB and the advantages of using MongoDB is that it is a document database which represents its data in BSON format. This allows the application to retrieve the data in JSON format. Document databases are extremely flexible as they allow variations in the structure of documents and allow storing of partially complete documents. The scale-out architecture allows one to develop scalable applications with growing data schemas without a need for normalizing the data. It can scale vertically and horizontally to handle large data sets.

The database achieves interoperability between the user and background data by having four primary schemas as of now are: a users schema, a messages schema, a conversation schema, and a members schema. The users schema contains data about users such as their first name, last name, email address, encrypted passwords, and id. The messages schema contains the data of who is the sender/recipient, what message was sent, what type of message was sent, the time at which the message was sent and any modifications to the message. The conversation schema contains data about what type of conversation it is (direct message, a group, a channel), the name of the conversation, who it was created by and what time it was created. The members schema contains data about the role of a given member, when they joined, when they read a message and if they are still permitted to send messages.

6 Issue and Concerns

The main concerns/issues I have faced during development thus far has been implementing SocketIO, the message broker and also understanding how to deploy my application on AWS.